Events, Outbox, Sagas & Background Jobs Cheatsheet

Lessons: 34 — Event-Driven & the Outbox · 35 — Sagas · 44 — Background Jobs & Queues

Examples: 34 · 35

Covers: async events, delivery semantics, the transactional outbox, sagas, task queues

Legend: `[*]` = concept or API the lessons have not covered yet

EVENT-DRIVEN BASICS

```
event                       a FACT, past tense: OrderPlaced, PaymentFailed
command                     a request, imperative: PlaceOrder — can be rejected
producer / consumer         publish without knowing who listens
topic / queue               fan-out to many vs work distributed among workers
event schema                id, type, version, occurred_at, payload, trace id
choreography                each service reacts to events; no coordinator
orchestration               one service tells the others what to do, in order
when to go async            slow work, fan-out, decoupling, spikes
when NOT to                 you need the answer in this request
```

EVENT PAYLOAD: NOTIFICATION vs STATE TRANSFER

```
notification                "OrderPlaced{ID: 42}" — just the fact and the id.
                            The consumer calls back to fetch what it needs.
                            + small, no stale data, no schema coupling
                            - a call per event; the producer must stay available
state transfer              "OrderPlaced{ID, Customer, Items, Total, ...}" — the
                            event carries everything a consumer could want.
                            + consumers are autonomous; no callback, works offline
                            - big messages, and every field is now a contract
event-carried state transfer the middle ground most systems land on: enough for
                            the common consumer, an id for the rest
the rule                    put in what a consumer needs to DECIDE, not everything
                            you happen to have
never send a whole DB row   it couples every consumer to your schema forever
ack order                   an ack means "I durably handled it", not "I received
                            it" — ack AFTER the work, or you lose messages on crash
```

DELIVERY SEMANTICS (memorize)

```
at-most-once                fire and forget; messages can be LOST
at-least-once               the real-world default; messages can be DUPLICATED
exactly-once                does not exist end-to-end — stop looking for it
what you actually build     at-least-once delivery + IDEMPOTENT consumers
idempotent                  handling the same event twice = handling it once
dedup key                   the event id, stored in a processed_events table
ordering                    guaranteed only per partition/key, if at all
poison message              keeps failing; move it to a dead-letter queue
```

THE DUAL-WRITE PROBLEM & THE OUTBOX

```
the problem                  save to the DB, then publish to the broker.
                             The process dies between the two. Now they disagree.
NOT a fix                    publishing inside the transaction (the broker isn't in it)
NOT a fix                    publishing first (the DB write may fail)

the outbox pattern
  BEGIN
    INSERT INTO orders ...
    INSERT INTO outbox (id, topic, payload, created_at) VALUES (...)
  COMMIT                     ONE transaction, one atomic decision

  a relay reads unsent outbox rows and publishes them
  SELECT ... FROM outbox WHERE published_at IS NULL
    ORDER BY id FOR UPDATE SKIP LOCKED LIMIT 100
  publish, then UPDATE outbox SET published_at = now()
  crash after publish, before update -> republished -> hence idempotent consumers

polling relay                simple, a few seconds of latency
CDC / logical replication [*] read the WAL instead of polling (Debezium)
LISTEN/NOTIFY             [*] Postgres push to wake the relay immediately
inbox pattern             [*] the mirror image on the consumer side
```

EVENT VERSIONING

```
add optional fields          the only safe change
never remove or rename       old consumers still read the old shape
version in the type          OrderPlaced.v2, or a version field
upcasting                [*] translate v1 -> v2 on read
tolerant reader              ignore unknown fields; don't fail on extras
publish both for a while     during a migration, then retire v1
```

SAGAS

```
the problem                  a business transaction across services; no 2PC
saga                         a sequence of LOCAL transactions, each with a
                             COMPENSATING action if a later step fails
orchestration                a coordinator holds the state machine and calls each step
                             — easy to debug, one place to look
choreography                 each service listens and reacts — fewer moving parts,
                             harder to see the whole flow
compensation ≠ rollback      you cannot un-send an email; you send an apology
  reserve stock  -> release stock
  charge card    -> refund card
  book the room  -> cancel the booking
idempotency key              per step, so a retry doesn't double-charge
the saga log                 persist the state after every step; resume after a crash
semantic locks           [*] mark the row "pending" so others see the in-flight state
timeouts per step            a step that never answers must fail the saga
(design the compensations FIRST — if a step can't be compensated, put it last)
```

BACKGROUND JOBS & TASK QUEUES

```
why out-of-process           the HTTP handler returns in ms; the work takes minutes
                             and must survive a deploy or a crash
202 Accepted + a job id      the API contract for async work
the enqueue                  inside the request; ideally in the same tx (or the outbox)
the worker                   a SEPARATE binary: cmd/worker/main.go, same domain code
asynq / Redis                enqueuer + workers, retries, scheduling, dead-letter
  asynq.NewTask("email:send", payload)
  client.Enqueue(task, asynq.MaxRetry(5), asynq.Queue("critical"))
  asynq.HandlerFunc(func(ctx, t *asynq.Task) error)   return err to retry
  srv.Run(mux)                   the worker's main loop
retry with backoff           exponential + jitter; a cap on attempts
dead-letter queue            after the last retry — alert on its depth
idempotent handlers          at-least-once again: the same job WILL run twice
visibility timeout           if the worker dies, the job returns to the queue
scheduled / periodic     [*] cron-style tasks, and delayed execution
priorities / weights     [*] separate queues so a bulk job can't starve a critical one
graceful shutdown            stop pulling, finish in-flight, then exit
DB-only alternative          a jobs table + SELECT ... FOR UPDATE SKIP LOCKED
queue vs outbox              outbox = "publish this fact reliably";
                             queue = "do this work reliably"
```

OBSERVABILITY FOR ASYNC WORK

```
propagate the trace id       put it in the message; restore it in the worker
queue depth                  the single most useful gauge
job age / oldest message     rising age = you are falling behind
retries and DLQ depth        alert on both
processing duration          a histogram per task type
log the job id everywhere    it's the request id of the async world
```

TRAPS & MEMORIZE

```
publish inside a transaction  the broker doesn't roll back — use the outbox
assuming exactly-once         then discovering duplicate charges in production
non-idempotent consumers      the #1 cause of async data corruption
no dead-letter queue          one poison message stops the whole consumer
retrying without backoff      you DDoS your own dependency
unbounded retries             a job that can never succeed retries forever
ordering assumptions          messages arrive out of order; design for it
compensations as an afterthought  some steps cannot be undone
the worker in the API process  a deploy kills in-flight jobs
big payloads in messages      send an ID; the consumer fetches what it needs
no visibility into the queue  depth and age are not optional
events used as commands       "OrderPlaced" that one service MUST handle is a command
```